Lecture 6: Combinational Logic Circuits

Lui Holder-Pearson

Aug 6, 2026

Learning Goals

- Understand the use within embedded systems, and design of:
  - Decoders and encoders;
  - Multiplexers and demultiplexers; and
  - Subtraction within an ALU.
- Understand how we use bus-wide combinational logic circuits to implement the control within computers.

Further Reading

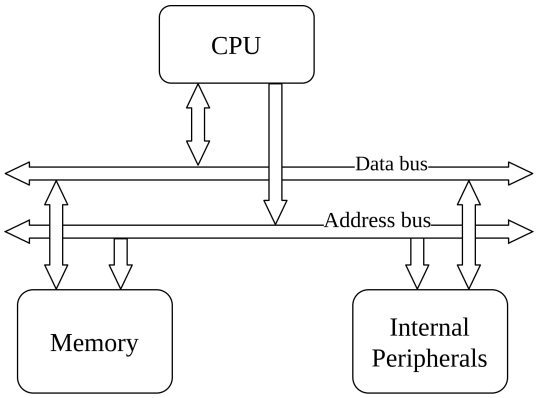
Rizzoni and Kearns. *Principals and Applications of Electrical Engineering*, Chapter 13.5
Youtube vid. similar to this lecture, but in a Scottish accent.

References

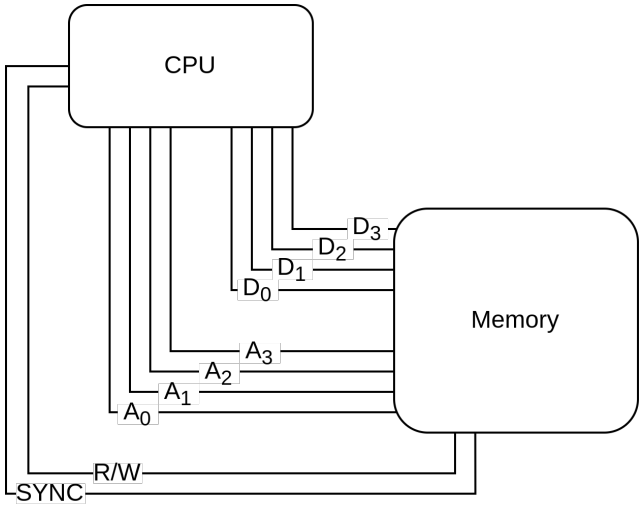
Ciaran Moore's past ENCE260 lecture notes
Rizzoni and Kearns. *Principals and Applications of Electrical Engineering*, Chapters 13 & 14.

1 Context: Address bus

How does the address bus work?



We know that the address bus has an address space defined as 2^n, where n is the number of individual connections in the address bus. We will often refer to the address as the binary number referred to by the address bus. For example, a 4-bit address bus can show address `0x0 - 0xF`.



Electrically, we know that each of these connections can either have a voltage present, or not: represented by `1` or `0`, and can be considered logical variables. The memory address is the sequence of logic variables, interpreted as a number.

2 Now what? Decoders and encoders

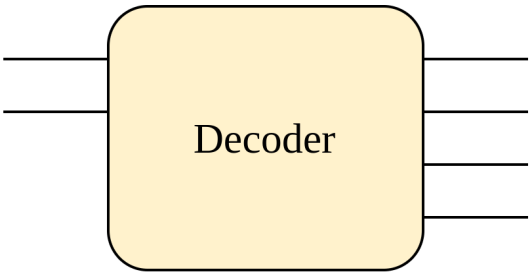
Fundamentally, memory is two-dimensional arrays of flip-flops[1]. The electrical signal within the data bus must connect to the appropriate row of flip-flops. Each flip-flop is capable of storing a single logical variable.

[1] Or equivalent, other types of memory.

But how specifically do we select the correct byte of memory to read the data from? We

Address	Bit number 7	6	5	4	3	2	1	0	
0x0									Word 0
0x1									Word 1
0x2									Word 2
...									
0xE									Word 14
0xF									Word 15

use **decoders** to select the row. A decoder takes N inputs, and has 2^N outputs.



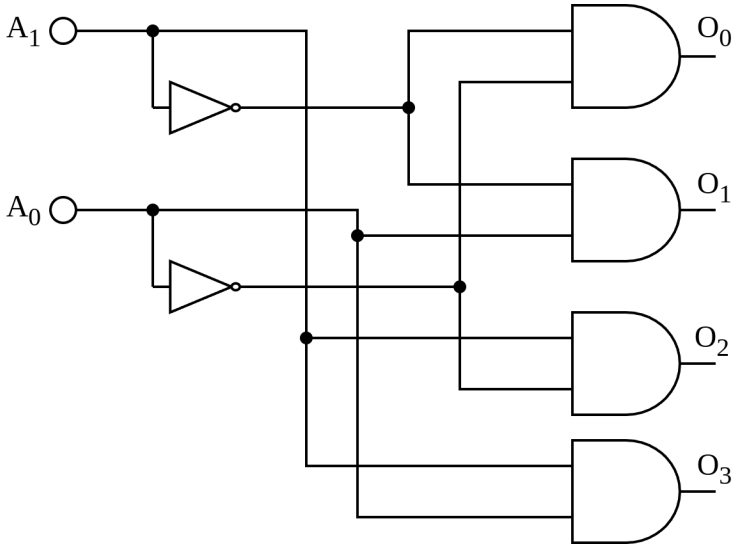
Exactly one output of the decoder is `HIGH` at any time; all others are `LOW`.

Inputs		Outputs			
A_1	A_0	O_3	O_2	O_1	O_0
0	0	0	0	0	1
0	1	0	0	1	0
1	0	0	1	0	0
1	1	1	0	0	0

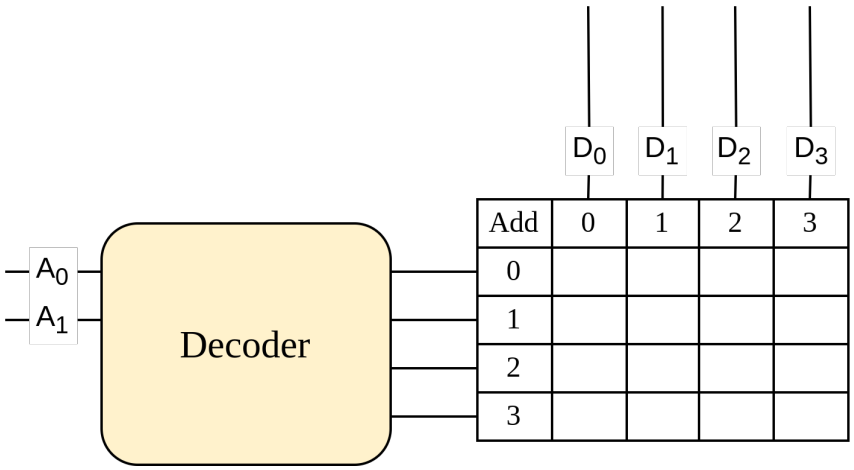
You can think of a decoder of converting a *logic sequence* which represents a number to a specific *logic variable*. The output logic value can be considered the answer to the question 'is the number encoded in the logic sequence equal to N?'.

As with everything* inside computers, decoders are made of logic gates. Similar to the adder circuits we saw in Lecture 5: CPU, we have multiple outputs from the same inputs, where each output encodes a specific minterm:

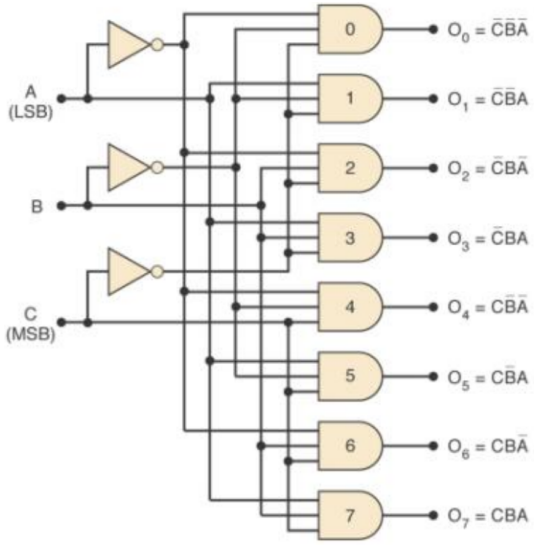
*mostly



Specifically, the output from the decoder is used to select the exact byte that is being read from:



How many outputs would a decoder with three inputs have?



2.1 Encoder

Simply put, an encoder is the inverse of a decoder. An encoder has 2^N inputs, *exactly* one of which is `1`, and encodes this into N outputs, as a number.

Inputs								Outputs		
I_7	I_6	I_5	I_4	I_3	I_2	I_1	I_0	A_2	A_1	A_0
0	0	0	0	0	0	0	1	0	0	0
0	0	0	0	0	0	1	0	0	0	1
0	0	0	0	0	1	0	0	0	1	0
0	0	0	0	1	0	0	0	0	1	1
0	0	0	1	0	0	0	0	1	0	0
0	0	1	0	0	0	0	0	1	0	1
0	1	0	0	0	0	0	0	1	1	0
1	0	0	0	0	0	0	0	1	1	1

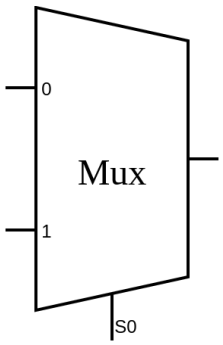
Exercise 1:

What would this logic circuit look like?

3 Multiplexers

A multiplexer (mux) takes multiple data inputs, and, based on control inputs, selects one as the output. The most simple multiplexer is a two-to-one (2:1) multiplexer:

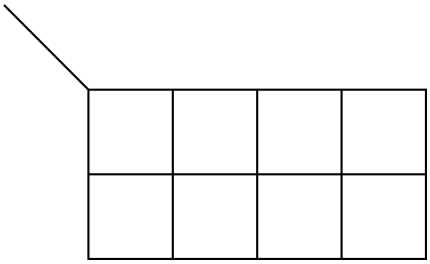
Input			Output
S	I_1	I_0	O
0	0	0	0
0	0	1	1
0	1	0	0
0	1	1	1
1	0	0	0
1	0	1	0
1	1	0	1
1	1	1	1



Alternatively:

Input			Output
S	I_1	I_0	O
0	X	0	0
0	X	1	1
1	0	X	0
1	1	X	1

We will determine the most efficient implementation using a Karnaugh map:



3.1 Alternative configurations

There are many other configurations of muxes:

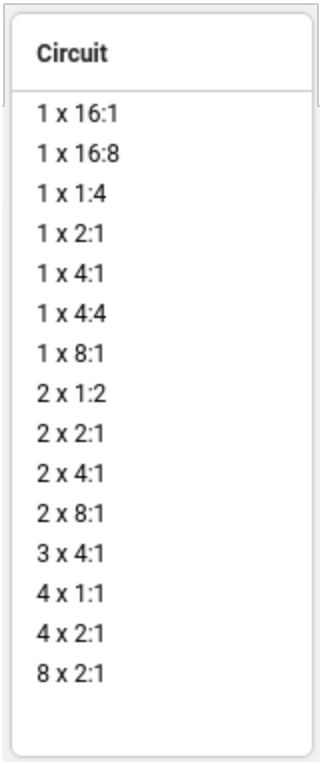


Figure 2: Multiplexers available from digikey.

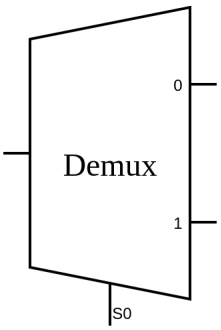
Exercise 2:

- Carry out the complete design process for a 4:1 multiplexer.
- Combine two 2:1 muxes to implement a 4:2 mux.

3.2 Demux

Demultiplexers do the opposite of a multiplexer. They take one digital input, and, as a function of some selector bits, direct it to one of several outputs. For example, the most basic, 1:2 demux:

Input		Output	
S	I	O_1	O_0
0	0	0	0
0	1	0	1
1	0	0	0
1	1	1	0



How could we simply implement a demux using a circuit we have already seen today?

4 Subtraction

We have seen how to implement an adder, but what about subtraction? As we saw in Tutorial 1, if we encode negative numbers in two's complement, we can simply add them together at face value. Thus, subtraction can be implemented by electrically implementing two's complement:

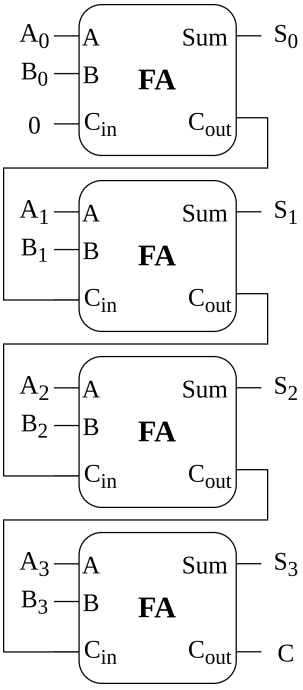
$$3 - 4 = -1$$

	0	1	0	0	↓ invert
	1	0	1	1	
+				1	↓ +1
=					-Subtrahend

	0	0	1	1	Minuend
+	1	1	0	0	
=					Difference

Inputs		Output
Sub?	*I*	*O*

Full cascaded 4-bit adder:



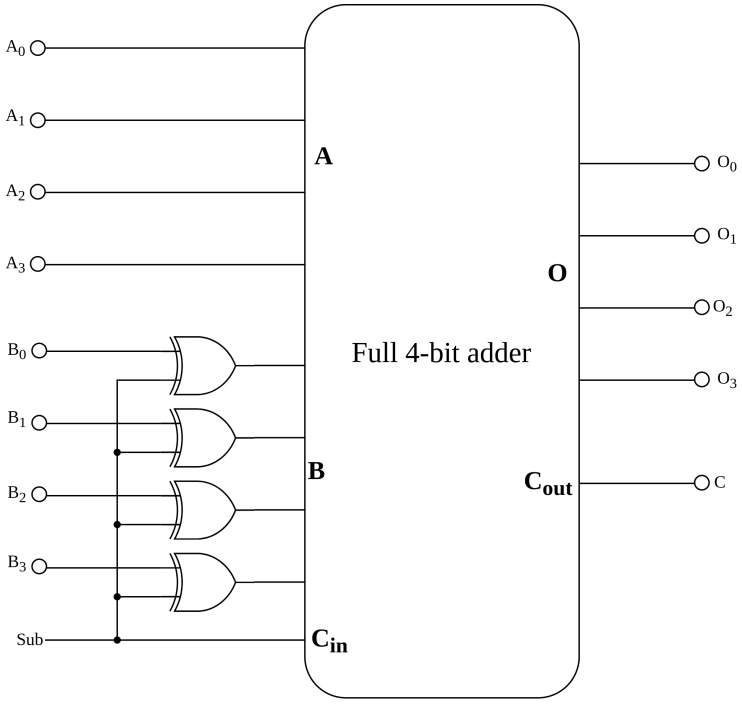
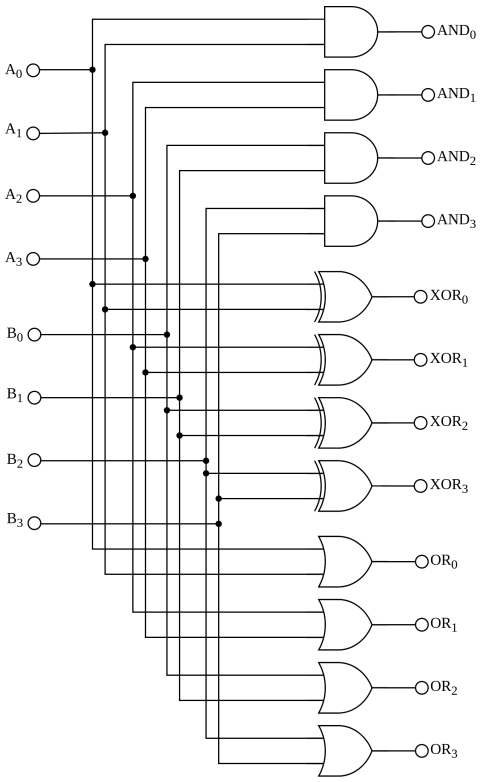


Figure 5: ALU with either addition or subtraction.

5 In practice

As we saw in Lecture 5 - CPU, the ALU is also capable of bit-wise Boolean operations. This is done by actually doing all of them, and then selecting the output:



Exercises:

1. What would the logic circuit for an 8:3 encoder look like?
2. Carry out the design process from a 4:1 multiplexer.
3. Implement a 4:2 mux using two 2:1 muxes.

Bonus: Implement a 2-bit ALU with addition, subtraction, bitwise AND, OR, XOR in circuit-simulator using only logic gates (similar to the NANDgame) - compare to my solution.

Summary

- Understand the use within embedded systems, and design of:
  - Decoders and encoders;
    → Decoders decode a n-bit binary number into exactly one true signal of 2^n possible outputs. They are used for selecting the byte in memory being read from, for example.
    → Encoders are the inverse, where exactly one of 2^n inputs are true, the corresponding number of the input is encoded as a binary number within a n-bit output.
  - Multiplexers and demultiplexers; and
    → Multiplexers are devices which map one of several inputs to the output. The most common configuration is Many-to-one, where n selection pins chose one of 2^n inputs to be connected to one output. Bus multiplexers also exist, where a collection of inputs is mapped to multi-bit output.
    → Muxes are used to select which specific operation is selected as the output from an ALU, for example.
    → A demultiplexer is the inverse: there is one single input (or multi-bit for bus demuxers) that is routed to one of several outputs. For example, to choose which peripheral device to communicate with.
  - Subtraction within an ALU.
    → Subtraction is implemented by converting the subtrahend into two's complement using an XOR gate on each bit of the subtrahend to invert it, and using the C_{in} property of the addition unit to add `1`.
- Understand how we use bus-width combinational logic circuits to implement the control within computers.
  → We have seen how decoders and muxes work to select the memory byte, and muxes to select the output of the ALU. We will see how this interfaces with software, and more specifics in Lecture 11 - Program Execution.

Next Lecture: Lecture 7 - Sequential logic circuits

6 Questions?